AgentForge Studio

The SRS Agent — interview to specification

A FastAPI + LangGraph service mounted inside AgentForge on port 7826 that interviews a non-technical customer one question at a time, turns the answers into a plain-English plan they approve, and composes an IEEE-830 SRS — with seven diagrams, a PDF and a builder handoff prompt — from that approved plan and nothing else.

Repository	https://github.com/Ravindu200232/full-stack-ai-web-developer
Branch	srs-agent
Scope	srs-agent/ and studio/components/srs/
Written	2026-08-14

Contents

1. Overview

The SRS agent is a self-contained FastAPI application that lives under `srs-agent/srs_agent/app/` and is run inside AgentForge's own process by a daemon thread (`srs-agent/srs_agent/mount.py` → `uvicorn` on 127.0.0.1:7826). The `app/` tree is a verbatim copy of a standalone srs-generator and is meant to stay that way: `srs-agent/srs_agent/bridge.py` collects every AgentForge-specific seam — the port, the staging directory `production-ready/.srs`, model routing through `agents.ollama_client`, and the Mongo URI from `agents.mongo` — and `app/config.py` is the only file in the copy that imports it. `bridge.py` explains why it is a plain import rather than environment variables: an env var only works if it is set before `app.config` is first imported, and nothing can enforce that ordering from inside a package importable from a server thread, a test or a script.

The product flow has three human gates, and the LangGraph assembly in `app/graph/workflow.py` mirrors them exactly: three small compiled graphs (`analysis` = intake → clarify|classify, `generation` = audit → generate_srs → diagrams, `customization` = customize → diagrams), each of which is a clean DAG because the human input that would make it cyclic happens between them. The interview itself is deliberately *not* a graph node — it is one question at a time, driven by the customer — so it lives in `app/agents/interview.py` and is stepped by `app/services/orchestrator.py`.

The single most load-bearing idea is that the **approved plan is a ceiling, not a summary**. `app/knowledge/plan_srs.py` composes the whole specification from `plan["records"]`, `plan["users"]`, `plan["screens"]`, `plan["features"]` and `plan["workflows"]` — nothing else. Its docstring names the bug that forced the inversion: the older domain-template composer (`app/knowledge/offline_srs.py`) never saw the app type, so every app got `users`, `roles`, `permissions`, `audit_logs`, RBAC and a marketing site, and "a calculator came out with five roles and fifteen API endpoints". `_auth_on()` in the same module decides once, from two independent votes (the app type's `auth_policy` and whether the plan names a non-anonymous role), and everything login-shaped is emitted only behind it. When the LLM is asked to enrich the skeleton, `_plan_scope_guard` in `app/agents/srs_generator.py` fails the response for naming a table the plan did not, feeding the offending names back into the repair loop.

Every LLM call degrades rather than fails. `app/llm/ollama_adapter.py` tries the primary model then the fallback, extracts JSON out of fences and prose (`app/llm/json_utils.py`), validates with a caller-supplied validator and re-prompts with the validation errors up to `llm_max_repair_attempts` (default 2). Above it, every agent catches `LLMUnavailable` and `LLMRepairFailed` and keeps its deterministic result: the interview falls back to `Topic.fallback_options` and a hardcoded question, the planner keeps `build_offline_plan`'s skeleton, the SRS keeps its skeleton, the diagram agent keeps the seven template Mermaid sources, and the customizer falls back to a regex-driven intent editor. The persistence layer does the same — `app/db.py` swaps a Motor store for an in-memory dict store when Mongo is unreachable, and `ensure_store()` (called from a middleware in `mount.py` every 5 seconds) heals a connection lost to AgentForge's mongod still booting, but refuses to upgrade a memory store that already holds answers.

2. What it does

Feature	What it does	Where
Intake from typed idea, PDF, image or voice	A project is created from an idea string; each attachment is saved to `<project>/uploads/`, read by whichever engine fits, and stored as an `extracted_sources` row. `build_brief` merges the idea and every source into one labelled brief that everything downstream reads.	`srs-agent/srs_agent/app/routers/projects.py` (`_ingest`), `srs-agent/srs_agent/app/extraction/brief.py`

Feature	What it does	Where
Per-page PDF reading — text layer where there is one, vision model where there is not	`read_pdf` extracts the text layer with pdfplumber then pypdf, marks any page whose text is thin or missing (`ocr_is_usable`), rasterises those with PyMuPDF at 144 DPI, and shows up to `MAX_MODEL_PAGES` (12) to the vision model three at a time. A twenty-page contract costs no model calls; a twenty-page scan gets read. Pages neither reader could read are listed by number in a `warning`.	`srs-agent/srs_agent/app/extraction/pdf_extract.py`
Images read by the model and by OCR, concurrently	`read_image` runs `describe_image` (base64 image on the `/api/chat` `images` key) and Tesseract in parallel. The model's reading leads; usable OCR text is appended because it is exact for a price or a code. With no model, OCR alone still carries a typed document; the raw `LLMUnavailable` string goes in `model_error`, not in the customer-visible `warning`.	`srs-agent/srs_agent/app/extraction/vision.py, srs-agent/srs_agent/app/extraction/ocr.py`
Audio transcription that never 500s	`FasterWhisperAdapter` loads a `base` model on CPU at int8 on first use; if faster-whisper is absent a `StubAdapter` returns a setup note and confirms the bytes were stored. `transcribe_audio` also catches failures *inside* an installed engine (missing CUDA, undecodable codec) and returns a warning, because raising loses the answer the customer just recorded.	`srs-agent/srs_agent/app/extraction/speech.py`
Nonsense gate before anything is fabricated	`looks_like_nonsense` scores the brief on recognised common words plus plausible-word heuristics (vowel ratio, keyboard-mash trigrams like `asd`/`qwe`, tripled letters). Failing it routes the analysis graph to `clarify_node` instead of classifying, so the flow starts from the app-type question rather than inventing an SRS.	`srs-agent/srs_agent/app/agents/intake.py, srs-agent/srs_agent/app/graph/workflow.py`
A deterministic interview backbone the LLM only words	`topics.build_queue` recomputes the whole remaining queue after every answer from an ordered `TOPICS` list, each entry gated by `applies_to` and `profiles`, some repeating over roles or tables. The model supplies wording and options only; a bad response costs a clumsy question, never a broken interview. `validate_question_budget` raises if any profile could exceed `MAX_VISIBLE_QUESTIONS` (25).	`srs-agent/srs_agent/app/knowledge/topics.py, srs-agent/srs_agent/app/agents/interview.py`
Questions conditioned on everything said so far	Each question's prompt carries `customer_context` (the raw idea, every sentence they typed themselves, and the text of every file they attached), the app-type pack, a digest of all prior answers, and the topic's intent plus its deterministic option list. Question N is genuinely conditioned on answers 1..N-1.	`srs-agent/srs_agent/app/agents/customer_context.py, srs-agent/srs_agent/app/agents/interview.py`
Prefill — asking back rather than asking twice	The model returns `known` (values the customer's own words already give) and `known_quote`. `_known_values` only keeps a value that matches an option actually on screen (or, on a typed question, something that could plausibly go in the box), so the UI can tick it with a "from what you said" badge and show the quote as "You said: …".	`srs-agent/srs_agent/app/agents/interview.py, studio/components/srs/Interview.jsx`
Clarification loop for typed answers	A typed answer that is not self-explanatory runs interpret → confirm meaning → ask purpose → confirm purpose. The raw text is never overwritten and `resolve()` returns nothing until both `meaning_confirmed` and `purpose_confirmed`. Clarification turns are excluded from the question count (`topics.is_clarification`).	`srs-agent/srs_agent/app/agents/clarify.py, srs-agent/srs_agent/app/services/orchestrator.py`
Attachments as interview answers	An answer can carry `attachments` (source ids). Extracted text is stored on the answer entry, flows into the answer digest and `customer_context`, and — when nothing was picked or typed — is promoted to the answer's value itself.	`srs-agent/srs_agent/app/agents/interview.py (`record`, `_attachment_entries`), studio/components/srs/Attachments.jsx`

Feature	What it does	Where
Coverage audit that names gaps rather than scoring them	`compute_coverage` marks each of 19 `COVERAGE_AREAS` covered by an explicit answer or by keyword evidence in the brief, weights optional misses at 0.6, and reports `critical_missing`. The planner then feeds the *named* gaps to the model with an instruction to state them in `assumptions` or raise them in `open_questions`.	`srs-agent/srs_agent/app/agents/coverage_auditor.py`, `srs-agent/srs_agent/app/knowledge/coverage_signals.py`, `srs-agent/srs_agent/app/agents/plan_generator.py` (`_what_nobody_asked`)
Plan generation with a depth floor	`build_offline_plan` always produces a complete plan from answers alone; the LLM then rewrites it in the customer's terms against `_plan_validator`, which is sized to the archetype — a 120-character `product_intent`, a purpose on every screen, 5 features, and (for non-thin archetypes) 2 records each keeping 2 things, one workflow of 2+ steps, and a role with 2+ `can_do` lines.	`srs-agent/srs_agent/app/agents/plan_generator.py`
Approval gate with real refusals	`plan_approval._verdict` refuses a superseded version, an already-approved plan, and any plan still carrying a required open question — and `state()` returns the same verdict, so the button is never offered for something that will be refused. Approval stamps the plan and moves the project to `plan_approved`.	`srs-agent/srs_agent/app/services/plan_approval.py`, `srs-agent/srs_agent/app/routers/plan.py`
SRS composed from the approved plan alone	`build_srs_from_plan` derives roles from `users`, tables and foreign-key relationships from `records`, public/protected pages from `screens`, and up to 40 functional requirements traceable to the plan — plus universal NFRs, an API surface, an RTM and branding. `merge_pack_planned` overlays the LLM's enrichment while dropping anything the plan does not contain.	`srs-agent/srs_agent/app/knowledge/plan_srs.py`, `srs-agent/srs_agent/app/agents/srs_generator.py`
Seven diagrams, template-first, LLM-improved, structurally checked	use_case, activity, sequence, erd, system_context, component and deployment Mermaid sources are built deterministically from the document and its plan; the LLM's versions replace them only if `mermaid_problems` finds nothing (balanced subgraphs, no orphan nodes, activate/deactivate parity, declared participants). `.mmd` is always written; `.svg`/`.png` render via a probed mermaid-cli.	`srs-agent/srs_agent/app/generators/diagrams.py`, `srs-agent/srs_agent/app/agents/diagram_generator.py`
Prompt-driven revision with versioning	`POST /customize` asks the model for a *patch* — only changed top-level sections — validates the merged result, folds list entries by identity so an edit replaces and an addition appends, bumps the minor version, snapshots the document and its diagrams, and records a `diff_summary` the UI shows in the revision rail.	`srs-agent/srs_agent/app/agents/customization.py`, `srs-agent/srs_agent/app/services/orchestrator.py` (`customize`)
Diagram regeneration after a revision	`plan_srs.effective_plan` projects the *current* document back into plan shape and writes it to `doc["effective_plan"]`; `diagrams._plan` prefers it over the frozen `approved_plan`, so a role or page added by prompt appears in the use-case, context, component and deployment diagrams even with the LLM unreachable.	`srs-agent/srs_agent/app/knowledge/plan_srs.py` (`effective_plan`), `srs-agent/srs_agent/app/generators/diagrams.py` (`_plan`)
Per-version artefact history	`storage.snapshot_diagrams` copies `.mmd` and `.svg` into `diagrams/v{version}/` before the next revision overwrites the fixed filenames, and rewrites the stored paths — so "show me v1.0.0" no longer shows v1.2.0's pictures. PNGs are deliberately excluded (largest artefact, only the current PDF reads them).	`srs-agent/srs_agent/app/services/storage.py`

Feature	What it does	Where
IEEE-830 PDF with a native vector fallback	ReportLab renders a cover, a TOC with page numbers, section 2 restating the approved plan in the customer's words, then the formal sections and a diagram appendix. Each diagram uses its rendered PNG, else a ReportLab vector drawing of the same data, else the Mermaid source in a code box — so the PDF always contains pictures without mermaid-cli or Chromium.	`srs-agent/srs_agent/app/generators/pdf.py, srs-agent/srs_agent/app/generators/diagram_draw.py`
Live agent console over SSE	Every agent emits to an in-process bus that persists to `agent_events` and fans out to subscribers; a subscriber first replays the last 200 events, then streams, with a 15-second `ping` so an idle connection is not mistaken for a dead one. Prompt traces and errors get their own channels and endpoints.	`srs-agent/srs_agent/app/services/events.py, srs-agent/srs_agent/app/routers/events.py`

3. Technology and tools

Technology	Version	What it does here	Why
FastAPI	—	The whole HTTP surface: eight routers under `app/routers/` plus the job shim, mounted on one app with a lifespan that connects the store and a CORS middleware.	Async end to end, and its ASGI app object is what makes the in-process job shim possible — `httpx.ASGITransport` replays a request into the same app with no socket.
Uvicorn	—	Serves the FastAPI app from a daemon thread inside AgentForge's process, `log_level="warning"`, `access_log=False`, `timeout_keep_alive=300`.	`Server.run()` calls `asyncio.run()`, so it brings its own loop — which is exactly why it needs its own thread: AgentForge's websocket server owns the main thread's loop.
LangGraph	—	Three `StateGraph`s over a shared `AgentState` TypedDict — analysis (with a conditional edge on `is_nonsense`), generation, customization — compiled lazily and cached in module globals.	Each graph maps to a phase separated by human input, which keeps every one of them a clean DAG.
Pydantic + pydantic-settings	—	`Settings` reads model, Mongo URI and storage dir with `model_post_init` filling them from `bridge`; `SrsEnvelope`/`SrsDocument` is the strict validation target the JSON repair loop validates against.	Leaf models are `extra="allow"` so the LLM may enrich them, while field validators still enforce ≥3 functional requirements, ≥3 NFRs and ≥1 role.
Ollama (via AgentForge's client)	—	All text and vision generation, over `POST {base}/api/chat` with `format: "json"`, `temperature: 0.2` and a measured `num_ctx`.	`bridge.route()` delegates to `agents.ollama_client._default_client().route(model)` so `:cloud` models reach ollama.com with the bearer token AgentForge already holds — the copied adapter posted every model to localhost:11434 and so only worked after a separate `ollama signin`.
httpx	—	The Ollama client, and the ASGI transport the job shim uses to replay requests in-process with `timeout=None`.	Async, and its `ASGITransport` needs no socket and no second port.
MongoDB via Motor	—	Thirteen collections (`projects`, `extracted_sources`, `question_sessions`, `answers`, `plans`, `srs_versions`, `diagrams`, `agent_events`, `prompt_traces`, `errors`, …) behind a tiny async `Store` protocol.	Points at AgentForge's own mongod on whatever port it settled on (`agents.mongo.MONGO.port`), with an in-memory dict store as a fallback so the app runs in CI and demos.
sse-starlette	—	`EventSourceResponse` for `/projects/{id}/events/stream`.	The studio's console needs a live feed; the generator emits a `ping` on a 15-second `asyncio.wait_for` timeout to keep it alive.
ReportLab	—	The SRS PDF (`BaseDocTemplate` with header/footer chrome and a `TableOfContents` fed by `afterFlowable` notifications) and the native vector diagrams drawn with `Drawing`/`Group`/`Rect`/`Polygon`.	Pure Python, so the PDF works on Windows with no native libraries and no browser.
pypdf / pdfplumber	—	PDF text-layer extraction, pdfplumber first then pypdf, both optional.	An exported invoice or typed spec has a real text layer, and extracting it is exact, instant and better than showing a picture of the page to a model.

Technology	Version	What it does here	Why
PyMuPDF	—	Rasterises the pages a text layer could not read, at `RENDER_DPI = 144`, so the vision model can read them.	Without it a scanned PDF extracts to nothing — the customer attaches their entire requirements document and the intake learns nothing.
pytesseract + Pillow	—	Exact OCR alongside the vision model; `tesseract_cmd()` honours `TESSERACT_CMD` then PATH then a list of known install paths, with easyocr as a heavier fallback.	It needs no model and no network, so intake keeps working on typed documents with Ollama stopped.
faster-whisper	—	Local speech-to-text, `WhisperModel("base", device="cpu", compute_type="int8")`, cached on the class after first load.	Selected automatically by `_select_adapter()` when importable; otherwise a stub returns a setup message rather than a fake transcript.
mermaid-cli (mmdc / npx)	—	Renders each `.mmd` to `.svg` (`-s 1`) and `.png` (`-s 2`) on a white background, with a 180-second per-render timeout.	Discovered by probing `--version` and believing the exit code — `shutil.which` is not enough, because a removed global install leaves its launcher shim behind and every render then dies with MODULE_NOT_FOUND.
Next.js 16 + React 19 (studio)	—	`Interview.jsx`, `PlanReview.jsx`, `SrsReview.jsx`, `views.jsx`, `Attachments.jsx` and `SrsResult.jsx` under `studio/components/srs/`, with `lib/srs-view.js` assembling one view object from five endpoints.	Shares an origin with AgentForge on purpose, so the preview iframe stays reachable — which is also what forces the job shim.
Tailwind CSS 4 + lucide-react	—	All studio styling and icons.	Repo-wide convention; the SRS panes use the same `Panel`/`Button`/`Table` primitives as the rest of the studio.

4. How it works

1. 1 — Start the service

`server.py` inserts `srs-agent/` on `sys.path`, starts `start_srs_api()` on a daemon thread, which imports `srs_agent.mount` and calls `mount.serve(port=7826)`. `mount.serve` attaches the job router, installs a middleware that calls `db.ensure_store()` at most once every 5 seconds, and runs uvicorn. Import failure and runtime crash are recorded separately in `SRS_API` and reported by `/api/srs-status`; neither can stop AgentForge starting.

2. 2 — Create the project and ingest attachments

The studio's "Plan it first" saves `srs_model`, then `POST /projects {idea}` → `orchestrator.create_project` writes a `projects` row with status `intake`. Each attachment goes to `POST /projects/{id}/inputs-json` as base64 (≤ `MAX_UPLOAD_BYTES` = 7,500,000, sized to the Next rewrite's 10 MiB body limit with base64's 4/3 inflation), is written to `<project>/uploads/`, routed by mode/content-type/extension to `read_pdf`, `read_image` or `transcribe_audio`, and stored as an `extracted_sources` row with the engine metadata as `meta`.

3. 3 — Analyze

`POST /projects/{id}/analyze` builds the brief (`build_brief(raw_idea, sources)`), runs the analysis graph: `intake_node` detects language by Unicode block and scores nonsense on `_content_only(brief)`; the conditional edge routes to `clarify_node` or `classify_node`. `classify_node` takes `classify_domain` + `guess_app_type` as priors and asks the model to refine `domain_key`, `app_type`, `build_category` and confidence. On any exception the orchestrator falls back to `_fallback_analysis`, which re-derives the classification with no LLM and no graph.

4. 4 — Open the interview session

`_new_interview_session` records `guessed_app_type`, its confidence and the classifier's `build_category_why`; `interview.ask` is called once and the question is cached on `session["current"]` by `_remember` — without that cache, remounting the interview screen would fire a fresh LLM call and reword the same question. The project moves to status `questioning`.

5. 5 — Ask and record, one question at a time

`topics.build_queue(session)` returns everything still to ask; the head item's topic is looked up and either answered from a fixed string (`app_type`, `extra_notes`) or sent to the model with the customer's own words, the pack, the answer digest and the topic brief. `_question` emits both shapes at once — the `Question` contract the coverage auditor and SRS digest speak, and the chips/index/prefill fields the UI renders.

6. 6 — Answer, clarify, re-derive

`POST /projects/{id}/interview/answer` resolves attachment ids to sources, then either advances an open clarification or calls `interview.record`. Recording `app_type` sets `session["app_type"]` and rebuilds `session["pack"]` via `catalog.build_pack`, which is what re-shapes the queue. A typed answer that fails `clarify.needs_clarification` opens a clarification state; a confirmed one is folded back with `_apply_clarification`, which keeps `raw` beside the confirmed text.

7. 7 — Finish the interview

When `_next_question` returns None the session is marked `complete`, `compute_coverage` runs over the brief, transcript and flat answers, the score and full coverage dict are written to both the session and the project, and the project moves to status `planning`.

8. 8 — Generate the plan

`POST /projects/{id}/plan` builds `build_offline_plan` as the skeleton, then prompts the model with: the customer's own words, the app type and domain, what the classifier already worked out, the domain library as "a menu, not a checklist", the answer digest (question → answer, plus what they typed and what they attached), the named coverage gaps, their unprompted end-of-interview notes, and the skeleton itself. `_merge` keeps the model's non-empty sections over the skeleton's. The plan is saved with a `content_hash`, a version number and `change_request`.

9. 9 — Revise or approve

A revision re-runs step 8 with `previous` and `revision` appended and `_REVISION_TAIL` ("a revision is not a re-plan"), producing version N+1. `POST /plan/approve` runs `plan_approval.approve`, refusing a superseded, already-approved or question-blocked plan; success stamps `approved_at` and sets project status `plan_approved`.

10. 10 — Generate the SRS

`POST /generate-srs` loads the approved plan (falling back to the latest) and runs the generation graph. `audit_node` recomputes coverage; `generate_srs_node` builds the skeleton with `build_srs_from_plan`, validates it, asks the model for an enrichment pack under `_plan_scope_guard`, merges with `merge_pack_planned`, revalidates, then attaches `approved_plan_markdown` and the builder handoff; `diagram_node` builds seven template diagrams, replaces any the model improved and validated, and renders them off-thread.

11. 11 — Persist the version

The orchestrator bumps the minor version (`1.0.0` → `1.1.0`), writes `srs_v{version}.json` and `srs_latest.json`, deep-copies the document and rewrites its diagram paths through `snapshot_diagrams` into `diagrams/v{version}/`, saves an `srs_versions` row, replaces the `diagrams` rows, and sets project status `generated`.

12. 12 — Revise the specification by prompt

`POST /customize` runs the customization graph. `customize_node` shows the model the document minus the derived sections, capped at `_VIEW_BUDGET` (12000 chars), and validates that `merge_edit(srs, patch)` still passes `validate_srs`. `_clean_patch` folds each returned section onto the current one; `customize_node` then writes `effective_plan` and `diagram_node` redraws. The orchestrator bumps the version, re-attaches the handoff, snapshots and labels the version `Customized: <prompt>`.

13. 13 — Hand off to the builder

The studio's approve button POSTs `/projects/{id}/approve`, reads `/builder-handoff`, and passes `handoff.prompt` back into `Home.startBuild` as the build prompt with `srs_id` attached. `server.py`'s `adopt_srs(srs_id, proj_dir)` copies `production-ready/.srs/<id>/` into `<project>/.agentforge/srs/` and additionally fetches four things the SRS writes on demand — the plan, the handoff, the interview and the PDF. The staging copy is deliberately kept so building twice from one SRS keeps working.

14. 14 — Enrich the architect's brief

`server.py`'s `_srs_brief` appends the full specification under the handoff prompt, because the prompt's 1200-word cap belongs to a different program's context budget and AgentForge's architect has neither limit. Diagrams, ambiguities and risks are left out — they are for the person reading the SRS tab.

5. The code, file by file

Path	What lives there
`srs-agent/srs_agent/mount.py`	Runs uvicorn on 7826 in a daemon thread, attaches the job shim, and installs the 5-second store-health middleware. Documents why the SRS needs its own asyncio loop.
`srs-agent/srs_agent/bridge.py`	The single seam to AgentForge: `SRS_PORT`, `SRS_STAGING`, `srs_model()`, `route(model)`, `num_ctx(model)`, `mongo_uri()`. Everything else in `app/` is a verbatim copy of the standalone generator.
`srs-agent/srs_agent/jobs.py`	Turns one slow POST into start/poll/read. Replays requests in-process over `httpx.ASGITransport`; keeps finished jobs 900s, caps at 200, and refuses `/jobs` paths. Its docstring carries the measurement: `generate-srs` returned 200 in 84.9s direct and 500 at exactly 30.0s through the Next rewrite.
`srs-agent/srs_agent/app/config.py`	The only file under `app/` that differs from the standalone copy. `model_post_init` fills the Mongo URI and model from `bridge`; `storage_dir` defaults to `production-ready/.srs`.
`srs-agent/srs_agent/app/graph/workflow.py`	Builds and lazily compiles the three LangGraph DAGs and explains why the interview is not one of them.
`srs-agent/srs_agent/app/services/orchestrator.py`	Every multi-step flow: analyze, interview_state, interview_answer, generate_plan, approve_plan, generate_srs, customize, project_detail. Owns versioning (`_bump_minor`) and the clarification fold-back.
`srs-agent/srs_agent/app/knowledge/topics.py`	The 940-line interview backbone — the ordered `TOPICS` list, `applies_to` conditions, per-profile question sets, `build_queue`, `total_estimate`, and the app-type option ordering with its `_mark_auth` warnings.
`srs-agent/srs_agent/app/knowledge/app_types.py`	Nine build categories with archetype, shell, palette, default pages/entities/operations, `PROFILE_CONTRACTS` (auth policy, required/prohibited pages), `guess_app_type` with `GUESS_FLOOR = 0.6`, and `build_pack` which fuses app type with domain.
`srs-agent/srs_agent/app/knowledge/domains.py`	The domain template library — hotel, retail, school, vehicle, hospital, restaurant plus a generic fallback — each with roles, modules, tables with real field lists, relationships, pages, workflows, notifications, reports, integrations, NFR focus and risks. Also `universal_tables()` and `classify_domain`.
`srs-agent/srs_agent/app/knowledge/plan_srs.py`	Composes the SRS from the approved plan and nothing else. `_auth_on`, `_tables_from_plan`, `_pages_from_plan`, `_requirements_from_plan`, `build_branding`, `merge_pack_planned` and the `effective_plan` inverse projection.
`srs-agent/srs_agent/app/knowledge/offline_srs.py`	The older domain-template composer, still used as the skeleton when no plan exists. `plan_srs`'s docstring names it as the thing that gave a calculator five roles.
`srs-agent/srs_agent/app/knowledge/coverage_signals.py`	Keyword signals per coverage area, so an idea that already says "guests pay online" is not marked as missing `payments_billing`.
`srs-agent/srs_agent/app/agents/interview.py`	The per-question LLM call, the answer digest, `_known_values` prefill filtering, `record` (which rebuilds the pack on `app_type` and promotes attachment text to the value), and `flat_answers`.
`srs-agent/srs_agent/app/agents/clarify.py`	The four-state clarification machine, `is_self_explanatory` as a deterministic pre-filter, duplicate detection against confirmed items, and `_clean_answer` which recovers a real name from a machine value.
`srs-agent/srs_agent/app/agents/customer_context.py`	Assembles the idea, every sentence typed during the interview, and the text of every attached file into one block every agent pastes into its prompt.
`srs-agent/srs_agent/app/agents/plan_generator.py`	`build_offline_plan`, the archetype-sized `_plan_validator`, the six prompt blocks (`_what_we_worked_out`, `_domain_library`, `_answers_digest`, `_what_nobody_asked`, customer notes, skeleton), `_merge`, and `render_plan_markdown`.

Path	What lives there
`srs-agent/srs_agent/app/agents/srs_generator.py`	Skeleton → enrichment → merge → validate, with `_plan_scope_guard` refusing tables outside the plan and, when there is no login, any requirement mentioning login/role/permission/admin.
`srs-agent/srs_agent/app/agents/customization.py`	Patch-and-merge editing: `_editable_view`, `_merge_list` folding by `_identity`, `_is_a_different_thing` / `_renumbered` for numeric ids, `_clean_patch`'s one-level dict walk, and the regex deterministic fallback `apply_customization`.
`srs-agent/srs_agent/app/agents/diagram_generator.py`	Builds templates, asks the model for richer Mermaid under `_validate_diagram_pack` (≥4 structurally valid), accepts per-diagram only what passes, and renders off-thread reporting every fault.
`srs-agent/srs_agent/app/generators/diagrams.py`	The seven Mermaid builders plus `actors_and_use_cases` (shared with the PDF renderer so the two cannot disagree), the structural checker `mermaid_problems`, the renderer probe `_works`/`_find_renderer`, and `render_diagrams`.
`srs-agent/srs_agent/app/generators/builder_brief.py`	The handoff: requirement list with `kind`/`entity`, mongoose-style models, pages, and the capped prompt string with forbidden-package sentences stripped. Its docstring records all three constraints and where in the builder's code they come from.
`srs-agent/srs_agent/app/generators/pdf.py`	The ReportLab report — cover, TOC, `_plan_section` restating the approved plan, sections 1-9, and the diagram appendix with its three-level image fallback.
`srs-agent/srs_agent/app/extraction/pdf_extract.py`	Per-page PDF reading with the vision fallback, the `VISION_AT_ONCE = 3` semaphore, and explicit reporting of pages neither reader could read.
`srs-agent/srs_agent/app/db.py`	`MemoryStore` and `MotorStore` behind one protocol, `connect_store`, and `ensure_store` with its two rules — replace an unpingable Motor store, upgrade a memory store only when it is empty.
`studio/components/srs/Interview.jsx`	The one-question-at-a-time UI: chips with `suggested` and `from what you said` badges, multi-select, the type-it-instead escape, attachments uploaded before the answer, and the guard that refuses to post when the file *was* the answer and none of it arrived.
`studio/components/srs/SrsReview.jsx`	Three-column review: revision rail with the prompt box and diff thread, nine document views from `views.jsx`, counts panel, PDF link, and the approve button that fetches `/builder-handoff` and hands its prompt to the build.

6. Why it is built this way

Design decisions with their reasons. Where the source records the failure that forced a choice, that failure is given rather than a general principle.

Decision	Why
The approved plan is a hard ceiling on the SRS, not a summary of it	`offline_srs.build_offline_srs` built from the domain template and never saw the app type, so every app got `users`, `roles`, `permissions`, `audit_logs`, RBAC and a public marketing site whether anyone asked or not — "a calculator came out with five roles and fifteen API endpoints" (plan_srs.py). `plan_srs` inverts it: every role, screen, record and requirement traces to something the customer signed off.
`guess_app_type` returns `("other", 0.0)` when nothing matches, and a lone single-word hit scores only 0.58 — below `GUESS_FLOOR = 0.6`	The old pair was `(DEFAULT_APP_TYPE, 0.0)` — a category named alongside an admission there is no evidence for it — and `record` read the first half and dropped the second. On a labelled set of 49 ordinary ideas, 35 matched nothing and every one was filed as "saas". Confidence used to reach 0.97 on one token if no other category scored, so "a barber shop" came back ecommerce on the word "shop" and a photo studio came back landing on "studio" — which switches the login off.
The app-type question orders the options by the guess but never pre-selects one, and every option's hint says what happens in that kind of app — with "nobody logs in" appended where `auth_policy` is disabled	`_app_type_options` used to ignore the session entirely, so every idea was offered "POS / Retail" first — simply the first key in the `APP_TYPES` dict — and a school library and a dental clinic both came out classified as POS. And this one answer sets `auth_policy` for the whole application while appearing to ask only what kind of thing it is: three categories short-circuit `_auth_on`, and nothing on screen said so.
The interview is not a LangGraph node; the queue is recomputed from scratch after every answer	It is one question at a time, driven by the customer, so it is stepped by the orchestrator rather than run to completion. Recomputing means answering "no auth" genuinely removes the role questions rather than skipping past them, and adding a table genuinely adds a field question for it.
The model supplies only question wording and options; the queue, conditions, answer storage and `options_locked` topics stay deterministic	A bad LLM response then costs a clumsy question, never a broken interview — every topic carries `fallback_options` and `_fallback_question` has a hardcoded sentence for each key.
A prefill only survives if it matches an option actually on screen	"A prefill the customer cannot trace back to something they said is worse than none — it ticks a box they never ticked." On typed questions the value only has to be something that could plausibly go in the box, and a number question rejects anything non-numeric.
A typed answer becomes a requirement only after both its meaning and its purpose are confirmed, and `raw` is never overwritten	A two-word note like "layaway" or "split bill" is in no option list, nothing downstream recognises it, and it can become a page, a table and a requirement on the strength of a guess. `is_self_explanatory` is a deterministic pre-filter so an unambiguous answer costs no round-trip — and every avoided call is an avoided chance to mangle something already right.
The plan validator has a real depth floor, sized per archetype	The old floor — non-empty intent, one named screen, three features — passed a plan with `records: [{"name": "Product", "keeps": []}]` and no workflows. Because `_plan_scope_guard` rejects any table the plan did not name, a thin plan cannot be recovered from later, so the repair loop must fire at plan time. Thin archetypes (landing, single-tool) legitimately have no records and are exempt.
Customization asks for a patch of changed sections, not the whole SRS	The document is far larger than the prompt budget, so the model only ever sees the first slice and any section it never saw came back missing. Patch-and-merge makes a truncated view harmless: what the model cannot see, it cannot drop.

Decision	Why
`_merge_list` folds patch entries by identity and never shrinks a section — except when the prompt itself asked to remove something	Three separate measured losses. A section returned holding only the touched entries is indistinguishable by length from an echo of the truncated copy, and dropping the patch threw the edit away. A `database_design` patch with three tables replaced all seven on a bakery SRS, losing `users`, `roles`, `baked_goods`, `orders` and `order_lines` while the diff summary said only that a table had been added. And "add a Warranty Claims page" came back as `FR-026`, overwriting "let a user edit and remove a sale record" — so `_is_a_different_thing` treats a bare number as a slot, not a name, and `_renumbered` gives the newcomer the next free id.
`effective_plan` projects the live document back into plan shape after a revision	Four of the seven template diagrams read the plan rather than the document, and `approved_plan` is frozen by design — so a role added by prompt never appeared in the use-case diagram and a page added by prompt never appeared in the component or deployment one. With Ollama up the enrichment pass papered over it; with templates alone — which ships whenever Ollama is unreachable — the diagram silently reverted. `product_intent` and `look_and_feel` still come from the approved plan, because nothing can recover the customer's phrasing.
Mermaid is validated structurally, not just by its first line, and the renderer is probed by running it	The checker counts subgraph/end pairs, orphan nodes, activate/deactivate parity and undeclared participants, so an LLM diagram is only accepted if it will actually draw. And `shutil.which("mmdc")` succeeds for a removed global install whose launcher shim remains, after which every render dies with MODULE_NOT_FOUND — silently, because the caller swallowed it. `_works` runs `--version` with a generous 120s timeout (npx may download the package) and believes the exit code.
Every slow POST goes through a generic job shim rather than an endpoint-by-endpoint list	The Next rewrite abandons a request at 30 seconds, and the cliff is not specific to `generate-srs`: a single interview answer was measured at 23.3s on a fast cloud model, and every LLM call scales with whichever model the user picked. An endpoint list would be a list of the ones that happened to be slow on this machine, on this day. It also forces uploads to travel as base64 JSON, since multipart cannot be replayed through the shim.
The store is re-checked before requests rather than connected once	AgentForge starts mongod in a thread at the same moment this app starts connecting, so the very first connection can lose the race and fall back to memory permanently — "an interview that reports success and is gone on restart". A memory store is only upgraded when it is EMPTY, because upgrading one with answers in it would silently discard them.

7. Interfaces

Name	Shape
`POST /projects`	{idea: str (min 1), language?: str} → {project: {id: "prj_…", title, raw_idea, status: "intake", current_version: "0.0.0", …}}
`POST /projects/{id}/inputs`	multipart: mode, text?, file? → {source, extraction, note}. `note` carries the engine's warning or error.
`POST /projects/{id}/inputs-json`	{mode, filename?, content_type?, data_base64?, text?} → same as /inputs. Accepts a browser `data:` URL prefix; 413 above 7,500,000 decoded bytes. This is the shape the job shim can replay.
`POST /projects/{id}/analyze`	→ {project, classification, needs_clarification, clarification_reason, question, questions}. Opens the interview session and returns question 1.
`GET /projects/{id}/interview`	→ {question, transcript: Question[], answers: [{question_id, value, raw_text}], done: bool, app_type}
`POST /projects/{id}/interview/answer`	InterviewAnswer {key, value?, text?, selected?: str[], custom?: str, attachments?: str[] (source ids)} → {question, done} or {question: null, done: true, coverage_score}
`GET /projects/{id}/questions`	→ {session, questions} — the raw transcript in the shape the rest of the API speaks.
`Question (emitted object)`	Two contracts at once: {id, question, why_needed, answer_type ∈ single_choice|multi_choice|yes_no|number|free_text, suggested_options, maps_to_srs_fields, coverage_areas, required} + {key, topic, subject, kind, index, total, options: [{label, value, hint?}], recommended, placeholder, optional, multiline, prefill, prefill_note}
`POST /projects/{id}/plan`	{revision?: str} → plan state. Empty revision builds v1; a non-empty one builds N+1 from the previous plan.
`GET /projects/{id}/plan`	→ {plan, markdown, version, versions: int[], content_hash, change_request, approved, approved_at, can_approve, reason, open_questions}
`Plan (JSON object)`	{app_name, product_intent, users: [{role, can_do[]}], screens: [{name, purpose, who[]}], records: [{name, keeps[]}], workflows: [{name, steps[]}], features[], look_and_feel, assumptions[], open_questions: [{question, required}], customer_notes}
`POST /projects/{id}/plan/approve`	{version?: int} → {approved: true, version, content_hash}, or HTTP 409 with the refusal reason (superseded / already approved / still to settle).
`POST /projects/{id}/generate-srs`	→ {project, version: "1.0.0"|bumped minor, summary: {functional, non_functional, use_cases, open_ambiguities, tables, roles, modules, diagrams}}
`GET /projects/{id}/srs-json`	→ {srs_document: {…}} validated against `SrsEnvelope`; includes `approved_plan`, `approved_plan_markdown`, `branding`, `builder_handoff`, `effective_plan` (after a revision) and `diagrams`.
`POST /projects/{id}/customize`	{prompt: str (min 1)} → {project, version, diff_summary: str[], summary}. 400 if no SRS exists yet.
`GET /projects/{id}/diagrams`	→ {diagrams: [{id, kind ∈ use_case|activity|sequence|erd|system_context|component|deployment, title, format: "mermaid", source, mmd_path?, svg_path?, png_path?, generated_by: "template"|"llm", svg?}]} — SVG is inlined when under 400,000 bytes.
`GET /projects/{id}/builder-handoff`	→ {appType, requirements: [{id: "R1", text, kind ∈ page|create|edit|delete|list|feature, entity}], models: [{name, fields, readOnly}], pages: [{route, name, primary}], prompt}. 404 for an SRS predating the approved-plan flow.

Name	Shape
`GET /projects/{id}/builder-prompt`	text/plain — just the prompt string, ≤1200 words and ≤8000 characters.
`GET /projects/{id}/download/{json|pdf}`	JSON: the SRS as an attachment named `<title>_SRS.json`. PDF: renders on demand via `build_srs_pdf`, status "Approved" when the project is approved or customized, else "Draft"; filename `<title>_SRS_v<version>.pdf`.
`POST /jobs and GET /jobs/{job_id}`	{path, method: "POST", body} → {job_id: "job_…", status: "running", path}; polling returns {status: running|done|error, http_status, result, error, elapsed}. Refuses relative paths and any `/jobs` path.